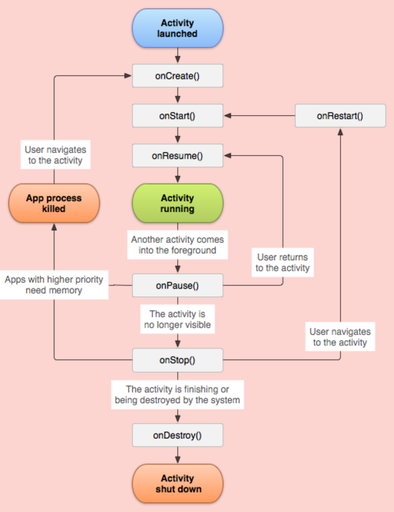
Lab 3 (App entrypoint)

Activity

Lifecycle



- **onCreate()**
  - Creating activity
  - - setContentView()
    - Init data
    - Init layout, view event
- **onStart()**
  - - Visible layout
    - Can't interact
    - Can (save data, make network call, query database)
- **onResume()**
  - Ready
  - - Handle core functions
    - Handle UI Logic
- **onPause()**
  - - User tap **Back, Recent** button
    - Partially visible
    - Lose focus
  - - Update UI (Not recommend)
    - **Don't** (save data, make network call, query database)
- **onStop()**
  - - No longer visible
    - Being Destroy
    - New activity start
- **onRestart()**
  - Just awakeness
- **onDestroy()**
  - - Before Destroy
  - - Remove event listener
    - Release resources

Activity State changes

- **Configuration change occurs**
  - Change between portrait and landscape orientation
  - Change language settings or input device
- **Handle multi-window cases**
  - When the user switches from app A to app B, the system calls **onPause() on app A** and **onResume() on app B**
- **Activity or dialog appears in foreground**
  - Dialog
    1. On Show
       - Case 1: Display partially
         - **only onPause() invoke in activity lifecycle**
       - Case 2: Display fullscreen
         - Like activity state
    1. On Dismiss
       - Call **onResume() invoke in activity lifecycle**
  - New Acitivty
    1. On Show
    - **Call onPause() -> onStop in Activity A**
    - **Call onCreate() -> onStart() -> onResume() in Activity B**

    2. On Return to activity A
    - **Call onPause() -> onStop() -> onDestoy() in Activity B**
    - Call ~~*onRestart()*~~ -> **onStart() -> onResume() in Activity A**
- **User taps or gestures Back**
  - - No calling **onSaveInstanceState()**
    Assumes the user taps Back with no expectation of returning to the same instance of the activity
  - - Handle logic **onBackPressed()** when you need
- **System or User kills app process**
  - - There is **no guarantee** that **onDestroy() is called**
    - The system n**eeds to free up memory** for a foreground app

Instant state

- **Save instant state**

```java
@Override
protected void onSaveInstanceState(@NonNull Bundle outState) {
    super.onSaveInstanceState(outState);
    outState.putString("myStringKey", myStringVariable);
    outState.putInt("myIntKey", myIntVariable);
}
```

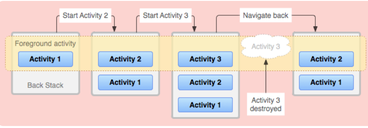
onSaveInstanceState(Bundle)

- **Restore instant state**

```java
@Override
protected void onRestoreInstanceState(@NonNull Bundle savedInstanceState) {
    super.onRestoreInstanceState(savedInstanceState);
    myStringVariable = savedInstanceState.getString("myStringKey");
    myIntVariable = savedInstanceState.getInt("myIntKey");
    // Update UI elements with the restored data
}
```

onRestoreInstanceState(Bundle)

Back stack



Navigating between activities

- **startActivity()**

```kotlin
Intent intent = new Intent(this, SignInActivity.class);
startActivity(intent);
```

- **startActivityForResult()**

```java
ActivityResultLauncher<Intent> startForResult = registerForActivityResult(
    new ActivityResultContracts.StartActivityForResult(),
    new ActivityResultCallback<ActivityResult>() {
        @Override
        public void onActivityResult(ActivityResult result) {
            if (result.getResultCode() == Activity.RESULT_OK) {
                Intent intent = result.getData();
                // Handle the result from the intent
                String returnedString = intent.getStringExtra("result_key");
            }
        }
    });
```

Option 1: ActivityResultLauncher

```java
Intent intent=new Intent();
intent.putExtra("name","charan");
startActivityForResult(intent,45);
```

Option 2: startActivityForResult()

- **setResult()**

```java
Intent intent = getIntent();
intent.putExtra("key", value);
setResult(RESULT_OK, intent);
finish();
```

Intent

Object contain data message to communicate between app components

Properties

- **Component name**
- **Action and Data**

The primary pieces of information in an intent are:

- **action** -- The general action to be performed, such as `ACTION_VIEW`, `ACTION_EDIT`, `ACTION_MAIN`, etc.
- **data** -- The data to operate on, such as a person record in the contacts database, expressed as a `Uri`.

Some examples of action/data pairs are:

- `ACTION_VIEW` ***content://contacts/people/1*** -- Display information about the person whose identifier is "1".
- `ACTION_DIAL` ***content://contacts/people/1*** -- Display the phone dialer with the person filled in.
- `ACTION_VIEW` ***tel:123*** -- Display the phone dialer with the given number filled in. Note how the VIEW action does what is considered the most reasonable thing for a particular URI.
- `ACTION_DIAL` ***tel:123*** -- Display the phone dialer with the given number filled in.
- `ACTION_EDIT` ***content://contacts/people/1*** -- Edit information about the person whose identifier is "1".
- `ACTION_VIEW` ***content://contacts/people/*** -- Display a list of people, which the user can browse through. This example is a typical top-level entry into the Contacts application, showing you the list of people. Selecting a particular person to view would result in a new intent { `ACTION_VIEW` ***content://contacts/people/N*** } being used to start an activity to display that person.

- **Extras (Bundle data)**
- Category (additional information about the action to execute)

- `CATEGORY_DEFAULT`
- `CATEGORY_BROWSABLE`
- `CATEGORY_TAB`
- `CATEGORY_ALTERNATIVE`
- `CATEGORY_SELECTED_ALTERNATIVE`
- `CATEGORY_LAUNCHER`
- `CATEGORY_INFO`
- `CATEGORY_HOME`
- `CATEGORY_PREFERENCE`
- `CATEGORY_TEST`
- `CATEGORY_CAR_DOCK`
- `CATEGORY_DESK_DOCK`
- `CATEGORY_LE_DESK_DOCK`
- `CATEGORY_HE_DESK_DOCK`
- `CATEGORY_CAR_MODE`
- `CATEGORY_APP_MARKET`
- `CATEGORY_VR_HOME`

- Flags

`flags`	`int` : The desired flags. Value is either `0` or a combination of `FLAG_GRANT_READ_URI_PERMISSION`, `FLAG_GRANT_WRITE_URI_PERMISSION`, `FLAG_FROM_BACKGROUND`, `FLAG_DEBUG_LOG_RESOLUTION`, `FLAG_EXCLUDE_STOPPED_PACKAGES`, `FLAG_INCLUDE_STOPPED_PACKAGES`, `FLAG_GRANT_PERSISTABLE_URI_PERMISSION`, `FLAG_GRANT_PREFIX_URI_PERMISSION`, android.content.Intent.FLAG_DEBUG_TRIAGED_MISSING, android.content.Intent.FLAG_IGNORE_EPHEMERAL, `FLAG_ACTIVITY_MATCH_EXTERNAL`, `FLAG_ACTIVITY_NO_HISTORY`, `FLAG_ACTIVITY_SINGLE_TOP`, `FLAG_ACTIVITY_NEW_TASK`, `FLAG_ACTIVITY_MULTIPLE_TASK`, `FLAG_ACTIVITY_CLEAR_TOP`, `FLAG_ACTIVITY_FORWARD_RESULT`, `FLAG_ACTIVITY_PREVIOUS_IS_TOP`, `FLAG_ACTIVITY_EXCLUDE_FROM_RECENTS`, `FLAG_ACTIVITY_BROUGHT_TO_FRONT`, `FLAG_ACTIVITY_RESET_TASK_IF_NEEDED`, `FLAG_ACTIVITY_LAUNCHED_FROM_HISTORY`, `FLAG_ACTIVITY_CLEAR_WHEN_TASK_RESET`, `FLAG_ACTIVITY_NEW_DOCUMENT`, `FLAG_ACTIVITY_CLEAR_WHEN_TASK_RESET`, `FLAG_ACTIVITY_NO_USER_ACTION`, `FLAG_ACTIVITY_REORDER_TO_FRONT`, `FLAG_ACTIVITY_NO_ANIMATION`, `FLAG_ACTIVITY_CLEAR_TASK`, `FLAG_ACTIVITY_TASK_ON_HOME`, `FLAG_ACTIVITY_RETAIN_IN_RECENTS`, `FLAG_ACTIVITY_LAUNCH_ADJACENT`, `FLAG_ACTIVITY_REQUIRE_NON_BROWSER`, `FLAG_ACTIVITY_REQUIRE_DEFAULT`, `FLAG_RECEIVER_REGISTERED_ONLY`, `FLAG_RECEIVER_REPLACE_PENDING`, `FLAG_RECEIVER_FOREGROUND`, `FLAG_RECEIVER_NO_ABORT`, android.content.Intent.FLAG_RECEIVER_REGISTERED_ONLY_BEFORE_BOOT, android.content.Intent.FLAG_RECEIVER_BOOT_UPGRADE, android.content.Intent.FLAG_RECEIVER_INCLUDE_BACKGROUND, android.content.Intent.FLAG_RECEIVER_EXCLUDE_BACKGROUND, android.content.Intent.FLAG_RECEIVER_FROM_SHELL, `FLAG_RECEIVER_VISIBLE_TO_INSTANT_APPS`, android.content.Intent.FLAG_RECEIVER_OFFLOAD, and android.content.Intent.FLAG_RECEIVER_OFFLOAD_FOREGROUND

Type

- Implicit

```java
// Create the text message with a string.
Intent sendIntent = new Intent();
sendIntent.setAction(Intent.ACTION_SEND);
sendIntent.putExtra(Intent.EXTRA_TEXT, textMessage);
sendIntent.setType("text/plain");

// Try to invoke the intent.
try {
    startActivity(sendIntent);
} catch (ActivityNotFoundException e) {
    // Define what your app should do if no activity can handle the intent.
}
```

- Explicit

```java
Intent intent = new Intent(this, TargetActivity.class);
```

Take a photo with a camera app

Android delegates actions to other applications by invoking an `Intent`. This process involves three pieces: the `Intent` itself, a call to start the external `Activity`, and some code to handle the image data when focus returns to your activity.

Here's a function that invokes an `Intent` to capture a photo.

```kotlin
val REQUEST_IMAGE_CAPTURE = 1

private fun dispatchTakePictureIntent() {
    val takePictureIntent = Intent(MediaStore.ACTION_IMAGE_CAPTURE)
    try {
        startActivityForResult(takePictureIntent, REQUEST_IMAGE_CAPTURE)
    } catch (e: ActivityNotFoundException) {
        // display error state to the user
    }
}
```

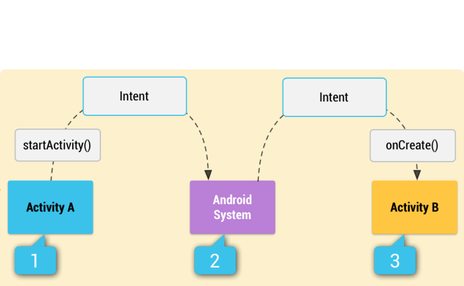
Intent camera

```java
@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    super.onActivityResult(requestCode, resultCode, data);

    if (requestCode == REQUEST_IMAGE_CAPTURE && resultCode == RESULT_OK) {
        Bundle extras = data.getExtras();
        Bitmap imageBitmap = (Bitmap) extras.get("data");  // <-- This is a thumbnai
        imageView.setImageBitmap(imageBitmap);
    }
}
```

Handle Result

How



Presented with xmind